Stochastic Anomaly Simulator

Project Documentation

Pole Anomaly Project

May 22, 2026

Contents

1 Introduction

This document provides a comprehensive overview of the **Pole Anomaly Project**, an anomaly detection and classification system for nuclear gamma dosimetry signals. The overall objective is to automatically identify and classify transient anomalous events—such as radiation spikes with distinct geometric signatures—embedded within continuous, noisy gamma dose rate measurements.

The project follows a two-phase approach. First, a **synthetic data generation pipeline** produces large-scale training datasets by injecting user-defined anomaly templates (Bell curves, Squares, M-shapes, Exponential ramps, etc.) into realistic simulated radiation backgrounds derived from real-world sensor baselines. Second, multiple **1D Deep Learning architectures (1D-CNN, ResNet, CNN-Attention)** are trained on these synthetic datasets to perform multi-class anomaly classification in a streaming inference setting.

The repository is organized into three main components:

- `data-gen/`: The synthetic data generation pipeline, including anomaly templates, the stochastic generator engine, and visualization scripts.
- `architectures/`: The neural network implementations, including the model architectures (1D-CNN, ResNet, CNN-Attention), training loop, inference engine, and robustness testing scripts.
- `documentation/`: This document and associated media.

2 Quick Setup Guide

This project isolates dependencies using a Python virtual environment to ensure absolute reproducibility.

2.1 Environment Setup

To initialize the project on a new machine, execute the following commands in the root of the project:

```
# 1. Create a Python Virtual Environment
py -3.14 -m venv .venv

# 2. Activate the Environment (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# 3. Install all dependencies
pip install -r requirements.txt
```

2.2 Dependencies

The `requirements.txt` installs the following libraries:

- `numpy` & `pandas`: Fast numerical array manipulation and CSV template handling.
- `torch` (PyTorch): Deep learning framework for the 1D-CNN. Installed from the PyTorch Nightly channel with CUDA 12.8 support via `--extra-index-url`.
- `plotly` & `kaleido`: Interactive HTML visualizations and static PNG export.
- `scikit-learn`: Utility functions for metrics and data splitting.
- `matplotlib`: Supplementary plotting library.
- `scipy`, `statsmodels`, `tslearn`: Statistical interpolation and legacy dataset compatibility.

Note: The `--pre` flag in `requirements.txt` enables installation of pre-release PyTorch builds, which is required for CUDA 12.8 support on newer GPUs (e.g., NVIDIA RTX 5060).

3 Synthetic Data Generation Pipeline (`data-gen`)

As the repository expands to include other projects (such as neural networks for anomaly detection), this section specifically details the anomaly generation pipeline.

3.1 File Structure Review

The repository enforces a strict separation of concerns, divided into modular directories:

- `data-gen/data/`: Contains raw background sensor readings (e.g., `2015_months_DebitDoseA.txt`) used as a real-world baseline, along with generated output datasets and the class mapping file (`class_mapping.txt`).
- `data-gen/anomalies/`: Contains the normalized `.csv` blueprints (templates) for the 6 anomaly shapes: `bell`, `bell_sq`, `fast_ascend`, `fast_descend`, `M`, and `square`.
- `data-gen/logs/`: The destination for Plotly's interactive HTML visualizations and PNG exports.
- `data-gen/`: Stores the pure mathematical generator library (`anomaly_generator.py`) alongside execution pipelines, real-data evaluators, and visualization launchers.

3.2 Explanation of the Engine and Scripts

3.2.1 `data-gen/anomaly_generator.py`

This is the core computational engine. It provides the `generate_anomaly()` function which transforms a normalized CSV blueprint into a physical discrete array. The engine mathematically scales the template temporally (period span) and vertically (amplitude intensity) while embedding Gaussian deformation using the limits declared in the CSV file.

3.2.2 `data-gen/evaluate_real_data.py`

This script analyzes the original physical gamma sensor data to extract baseline drift and noise characteristics. It calculates the moving average baseline to isolate high-frequency noise from the slow, low-frequency sensor drift. Crucially, it calculates the volatility σ of the Ornstein-Uhlenbeck (OU) process used for synthetic generation. Because a moving average heavily dampens step-by-step differences, the script applies a calibrated mathematical correction to σ (e.g., 0.037) so that the synthetic baseline explores the same dynamic range (15-27 units over 40,000 points) as the real 2015 sensors. The results are saved to `real_data_params.json`.

3.2.3 `data-gen/interactive_labeling_helper.py`

An interactive visualization tool that plots the real, unlabelled sensor data alongside its calculated baseline and a 3σ confidence threshold. It allows the user to hover over visual anomalies in the real data to measure their exact amplitude and period, which directly informs the configuration parameters for synthetic generation.

3.2.4 `data-gen/generate_custom_dataset.py`

The primary execution pipeline for training data. It utilizes a centralized `CONFIG` dictionary explicitly defining generation constraints (such as anomaly amplitude, period, and frequency) and seamlessly imports the real-world statistical parameters generated by `evaluate_real_data.py`.

Spanning datasets of up to 10,000,000 steps, it generates a highly realistic **wandering baseline** using an Ornstein-Uhlenbeck (OU) process.

The Wandering Baseline Logic (OU Process): Real-world sensors (like Gamma dose monitors) rarely maintain a perfectly flat baseline; they experience slow, low-frequency drifts over time due to environmental factors like temperature changes or sensor calibration drift. To mathematically simulate this, the generator employs the Ornstein-Uhlenbeck process, a mean-reverting random walk described by the discrete stochastic differential equation:

$$y_t = y_{t-1} + \theta(\mu - y_{t-1}) + \sigma\mathcal{N}(0, 1) \tag{1}$$

Where:

- μ (Global Mean): The long-term equilibrium value (e.g., 99.26).
- θ (Mean-Reversion Rate): Controls how strongly the drift is pulled back to the mean (e.g., 0.000004, ensuring very slow, realistic wandering).
- σ (Volatility): The scale of the random drift step.

This wandering baseline is then overlaid with standard high-frequency Gaussian noise. Finally, it randomly samples anomaly templates from the library, calculates randomized injection amplitudes and duration bounds, and injects them into the signal. Each injected shape is mapped to a unique multi-class integer ID, tagging absolute ground-truth labels alongside the continuous signal.

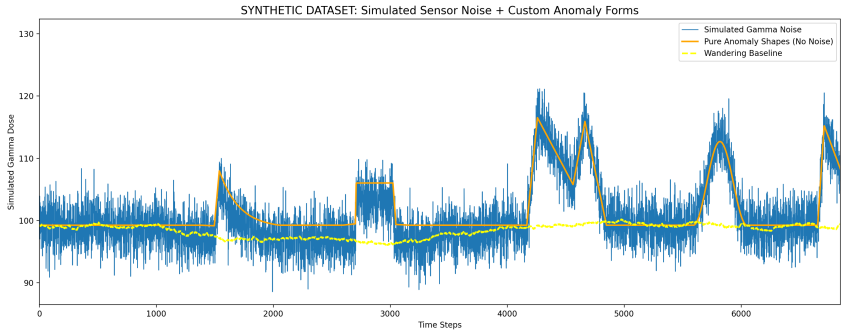

Figure 1: Output of generate_custom_dataset.py

3.2.5 `data-gen/test_generator_deformation.py`

A visual sandbox and boundary debugging tool. Iterating automatically over all discovered templates in the `data-gen/anomalies/` directory, it runs exactly 5 generation iterations of each template to visually verify and validate the variability limits enforced by the template's deformation parameters. Exports directly to HTML and PNG.

3.2.6 `data-gen/visualize_anomalies_templates.py`

This script visualizes the pure original mathematical blueprints. It renders the shapes and their interpolation properties exactly as described in the blueprint CSV, without injecting randomization or temporal noise.

3.3 Anomaly Template Creation

Instead of relying on rigid pre-recorded dataset distributions, targeted geometric structures are freely defined as CSV blueprints in the `data-gen/anomalies/` folder.

3.3.1 Configuration Blueprint (CSV Logic)

A standard anomaly template requires 7 strict column definitions mapping normalized coordinates and behavioral constraints:
`time, value, t_minus, t_plus, v_minus, v_plus, interp`

- **time & value**: Normalized shape coordinates (scaling from 0.0 to 1.0). They define the key inflection points of the curve.
- **Deformation Multipliers (t_minus, t_plus, v_minus, v_plus)**: Define spatial jitter limits per control point. A value of `1.0` allows symmetric Gaussian distortion up to 100% of the global variance parameter. A value of `0.0` locks the point rigidly in that direction.
- **interp (Interpolation Mode)**: Defines the mathematical connection between consecutive control points:
  - `linear`: Simple linear interpolation between points.
  - `exp_up`: Concave-upward exponential growth (slow start, rapid finish), with curvature controlled by $\alpha = 3.0$.
  - `exp_down`: Concave-downward exponential curve (rapid start, slow finish), the complement of `exp_up`.
  - `bell`: Cosine envelope ($\frac{1}{2} - \frac{1}{2}\cos(\pi x)$), generating a smooth S-curve suitable for Gaussian-like bell shapes.

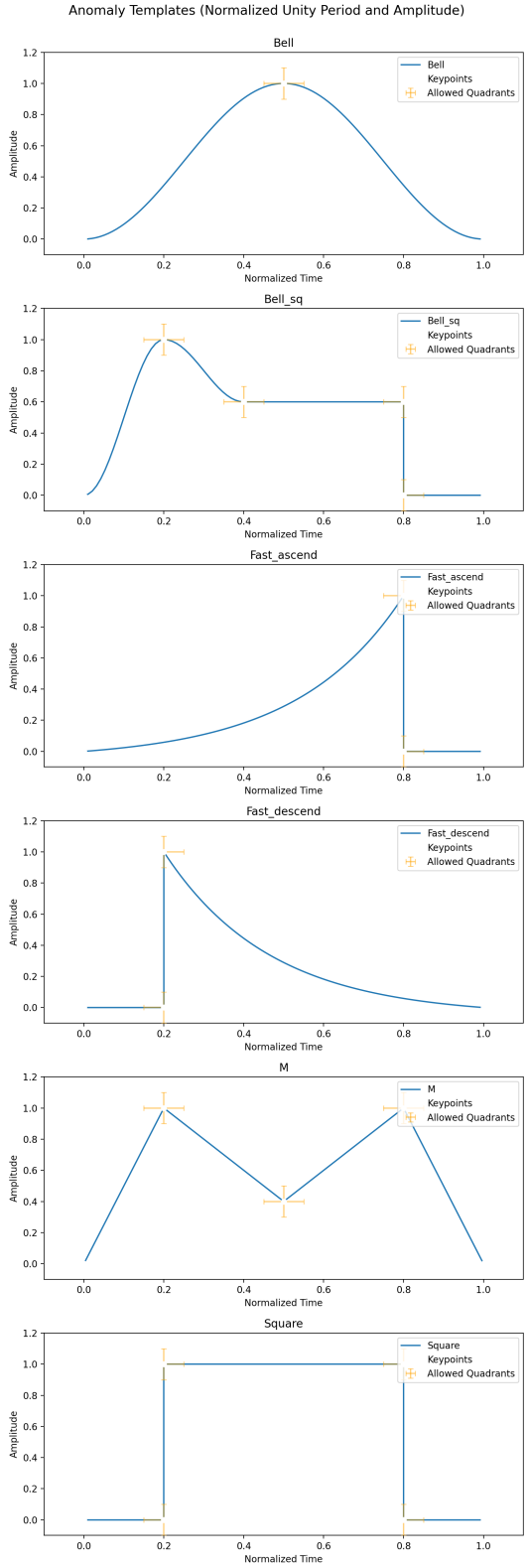

Figure 2: Output of visualize_anomalies_templates.py

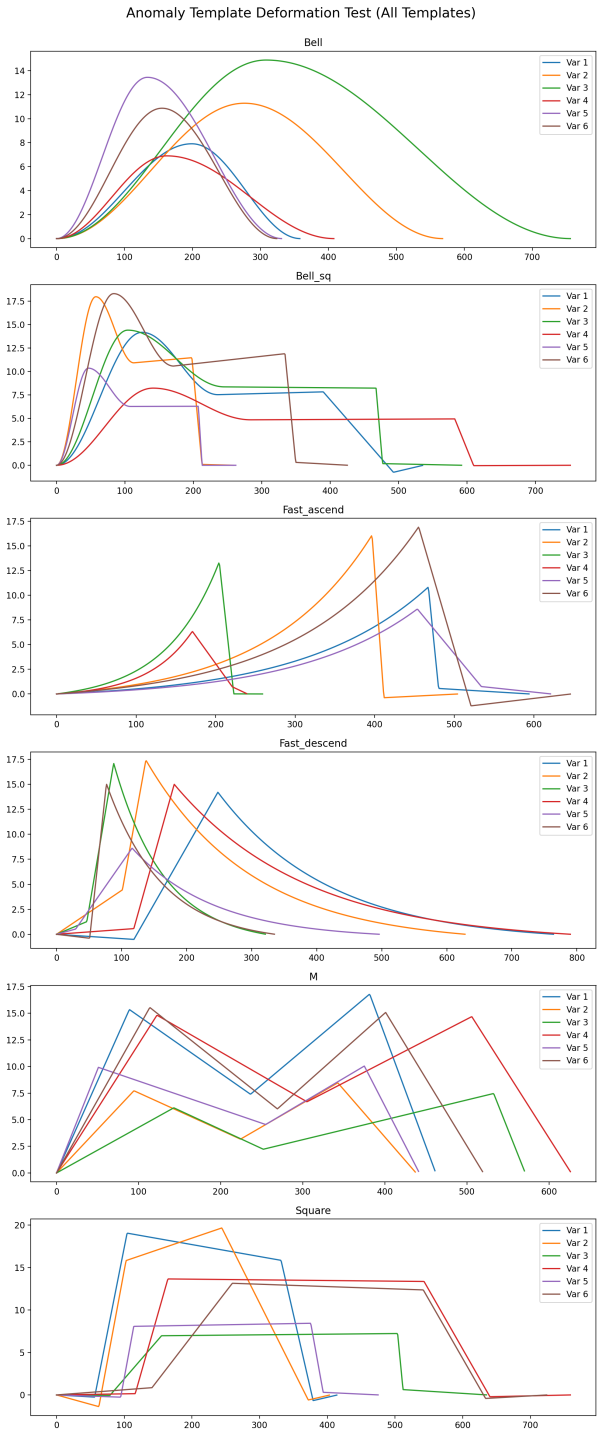

Figure 3: Output of test_generator_deformation.py

4 Inference and Evaluation Methodology

Anomaly detection models on time series data face a fundamental discrepancy between how they are *trained* and how their real-world performance is *evaluated*. This project employs two distinct evaluation paradigms: Window-Level Evaluation (used strictly during training) and Event-Level Evaluation (used during inference and robustness testing).

4.1 Window-Level Evaluation (Training Script)

During training (e.g., in `train.py`), the continuous time series is sliced into thousands of overlapping windows of a fixed size (e.g., 512 points). A window is labeled as an anomaly if anomalous points occupy more than a strict threshold (e.g., 10%) of the window's span.

F1 Score Penalty: Because the dataset relies on sliding windows, many generated windows capture only the extreme edges of an anomaly (e.g., 85% background noise and 15% anomaly curve). These "boundary" windows are highly ambiguous and often misclassified by the network. During training, the Macro F1 score evaluates each window independently. Consequently, the F1 score appears artificially deflated because the network is heavily penalized for hesitating on these blurry edges, especially under severe class imbalance conditions.

4.2 Event-Level Evaluation (Inference Script)

To provide a more practical and realistic assessment of model performance, the inference scripts (such as `visualize_inference.py` and `robustness_test.py`) employ Event-Level Evaluation.

Methodology:

1. **Accumulation:** As the sliding window passes over the continuous data, the model predicts the probabilities for each window. Since the windows overlap (stride of 10), each individual time step receives multiple probability predictions. These predictions are accumulated and averaged, creating a smoothed, continuous probability curve for the entire dataset.

2. **Event Grouping:** Instead of judging rigid windows, the script groups contiguous ground-truth anomaly points into a single discrete "event."

3. **Scoring:** An event is considered successfully detected and correctly classified if the averaged prediction curve peaks at the correct class at any point during the event's duration.

Impact on Metrics: By grouping points into events, the model is no longer penalized for boundary ambiguity. As long as the network's confidence correctly identifies the anomaly at its core/peak, the entire event is counted as a success. This leads to near 100% accuracy in inference, accurately reflecting the model's practical utility despite the deflated window-level F1 scores seen during training.

5 1D Convolutional Neural Networks: Theoretical Background

This section provides a comprehensive, ground-up theoretical foundation for 1D Convolutional Neural Networks (1D-CNNs). It is designed to provide sufficient mathematical rigor so that a reader could implement a 1D-CNN from scratch in C or NumPy, without relying on high-level libraries like PyTorch.

5.1 The 1D Convolution Operation (Forward Pass)

Standard fully-connected networks are ill-suited for time-series signal processing. They treat the input as an unordered set of independent variables, destroying temporal topology, and they lack **translation equivariance** (the ability to recognize a pattern regardless of where it occurs in time).

A 1D-CNN solves this by using a "sliding window" approach. Instead of learning a massive weight matrix for the entire signal, the network learns small templates called **filters** or **kernels**.

5.1.1 Single Channel Convolution

Let $\mathbf{x}$ be a 1D input signal of length T, and $\mathbf{w}$ be a learnable kernel of size K. The convolution operation slides the kernel $\mathbf{w}$ across the signal $\mathbf{x}$. At each time step t, it computes the dot product between the kernel and the local patch of the signal:

$$y[t] = \sum_{i=0}^{K-1} w[i] \cdot x[t+i] + b \tag{2}$$

where b is a learnable bias scalar. Mathematically, this dot product acts as a **pattern matcher**. If the signal patch $x[t \dots t+K-1]$ is perfectly aligned with the weights in the kernel w, the output $y[t]$ produces a massive positive activation. The resulting sequence $\mathbf{y}$ is called a **feature map**.

5.1.2 Multi-Channel Convolution

A single filter can only detect one specific shape (e.g., a rising edge). To detect complex anomalies, a layer must learn dozens of filters simultaneously. Furthermore, deeper layers in the network do not operate on the raw 1D signal; they operate on the multiple feature maps generated by the previous layer.

Let C_{in} be the number of input channels and C_{out} be the number of desired output channels (filters). The kernel is now a 3D tensor $\mathbf{W}$ of shape (C_{out}, C_{in}, K). The output feature map for a specific output channel c_{out} at time t is the sum of convolutions across all input channels c_{in}:

$$y_{c_{out}}[t] = \sum_{c_{in}=0}^{C_{in}-1} \sum_{i=0}^{K-1} W[c_{out}, c_{in}, i] \cdot x_{c_{in}}[t+i] + b_{c_{out}} \tag{3}$$

This operation preserves **local connectivity** (neurons only look at a small temporal patch K) and enforces **weight sharing** (the exact same tensor $\mathbf{W}$ is applied at every time step t), which drastically prevents overfitting and reduces computational cost.

5.2 Non-linear Activations

Without a non-linear activation function, stacking multiple convolutional layers would mathematically collapse into a single, linear convolution. After computing the feature map $\mathbf{y}$, we apply an activation function σ element-wise.

5.2.1 ReLU (Rectified Linear Unit)

The most widely used activation in CNN hidden layers is ReLU. It passes positive activations (where the filter "found" its target pattern) and zeroes out negative ones:

$$a[t] = \max(0, y[t]) \tag{4}$$

Its derivative is trivially 1 if $y[t] > 0$, and 0 otherwise. This simplicity prevents the vanishing gradient problem during deep backpropagation.

5.2.2 Softmax

At the very end of the network, after flattening the final feature maps into a vector of logits $\mathbf{z}$ (one for each class), we apply the Softmax function to convert them into a valid probability distribution:

$$\hat{y}_c = \frac{e^{z_c}}{\sum_k e^{z_k}} \tag{5}$$

5.3 Pooling Layers (Downsampling)

To hierarchically build larger receptive fields and reduce computational cost, CNNs use **Pooling layers** between convolutions.

MaxPooling slides a small window of size P (usually 2) across the feature map with a stride of P. Instead of a dot product, it simply outputs the maximum value in that window:

$$p[t] = \max_{i \in [P \cdot t, P \cdot t + P - 1]} a[i] \tag{6}$$

Intuitively, MaxPooling asks: *"Did this specific feature appear anywhere within this small region?"* If yes, it keeps the high activation and discards the exact spatial coordinate. This halves the length of the signal ($T \to T/2$) and provides **local translation invariance** against small signal jitters.

Crucially for implementation, during the forward pass, a MaxPooling layer must cache the **argmax index** (the exact position i that contained the maximum value) because gradients will only be routed back through that specific winning "pixel" during backpropagation.

5.4 The Loss Function

To train the CNN, we quantify its prediction error using **Cross-Entropy Loss**. For a single window, comparing the true one-hot label vector $\mathbf{y}$ against the predicted probabilities $\hat{\mathbf{y}}$:

$$\mathcal{L} = -\sum_k y_k \log(\hat{y}_k) \tag{7}$$

Proof: Derivative of Softmax with Cross-Entropy

A fundamental result in machine learning is how elegantly the gradient simplifies at the output layer. We want to find the error signal $\delta_j = \frac{\partial \mathcal{L}}{\partial z_j}$ where z_j is the pre-softmax logit. By the multivariate chain rule:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \sum_k \frac{\partial \mathcal{L}}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_j} \tag{8}$$

1. Derivative of Loss: $\frac{\partial \mathcal{L}}{\partial \hat{y}_k} = -\frac{y_k}{\hat{y}_k}$ 2. Derivative of Softmax (quotient rule): - If $k = j$: $\hat{y}_j(1 - \hat{y}_j)$ - If $k \neq j$: $-\hat{y}_k \hat{y}_j$

Substituting these into the sum:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \left(-\frac{y_j}{\hat{y}_j}\right)\left(\hat{y}_j(1 - \hat{y}_j)\right) + \sum_{k \neq j} \left(-\frac{y_k}{\hat{y}_k}\right)\left(-\hat{y}_k \hat{y}_j\right)$$

$$= -y_j(1 - \hat{y}_j) + \sum_{k \neq j} y_k \hat{y}_j = -y_j + y_j \hat{y}_j + \hat{y}_j \sum_{k \neq j} y_k$$

Since the true labels y must sum to 1, $\sum_{k \neq j} y_k = 1 - y_j$. Substituting this:

$$= -y_j + y_j \hat{y}_j + \hat{y}_j(1 - y_j) = -y_j + y_j \hat{y}_j + \hat{y}_j - y_j \hat{y}_j$$

The complex terms cancel out, leaving the beautiful final result for the output layer error:

$$\delta_j^{\text{out}} = \frac{\partial \mathcal{L}}{\partial z_j} = \hat{y}_j - y_j \tag{9}$$

5.5 Backpropagation in 1D-CNNs (Backward Pass)

Once the output error δ^{out} is calculated, the **Backpropagation algorithm** uses the chain rule to pass this error backward through the network, layer by layer, to compute the gradient of the loss with respect to every single filter weight.

5.5.1 Backpropagating through MaxPooling

Because MaxPooling only routed the maximum value forward, it acts as a gate. The error δ from the subsequent layer is routed entirely to the specific index that "won" the max pooling (which we cached during the forward pass). All other indices in the pooling window receive a gradient of 0, because changing them slightly would not have affected the forward output.

5.5.2 Backpropagating through 1D Convolution

This is the mathematical core of training a CNN. Let the error signal received from the next layer be $\delta[t] = \frac{\partial \mathcal{L}}{\partial y[t]}$. We must compute two sets of gradients for our single-channel convolution equation (Eq. 1).

1. Gradient with respect to the filter weights ($w[i]$): A single weight $w[i]$ in the filter was multiplied by the input $x[t+i]$ at *every* time step t as the filter slid across the signal. By the chain rule, we sum its contribution over all time steps:

$$\frac{\partial \mathcal{L}}{\partial w[i]} = \sum_t \frac{\partial \mathcal{L}}{\partial y[t]} \frac{\partial y[t]}{\partial w[i]} = \sum_t \delta[t] \cdot x[t+i] \tag{10}$$

This reveals a profound mathematical symmetry: to find how to update the filter weights, we perform a **cross-correlation** between the forward input signal x and the backward error signal δ. (For multi-channel convolutions, this cross-correlation is summed across all batch items and input dimensions).

2. Gradient with respect to the input ($x[t]$): To pass the error further backward to earlier layers, we need $\frac{\partial \mathcal{L}}{\partial x[t]}$. A single input point $x[t]$ affects multiple outputs y as the kernel slides over it. Specifically, it is multiplied by $w[0]$ to create $y[t]$, $w[1]$ to create $y[t-1]$, and so on up to $w[K-1]$ to create $y[t-(K-1)]$.

Summing these contributions via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial x[t]} = \sum_{i=0}^{K-1} \frac{\partial \mathcal{L}}{\partial y[t-i]} \frac{\partial y[t-i]}{\partial x[t]} = \sum_{i=0}^{K-1} \delta[t-i] \cdot w[i] \tag{11}$$

This is an elegant symmetry: to pass the error backward, we perform a **full convolution** of the error signal δ with the **flipped** kernel w. This allows deep learning frameworks to reuse highly optimized forward convolution algorithms for the backward pass.

5.6 Optimization and Regularization

5.6.1 Adam Optimizer

With the gradient of the weights $\nabla_{\mathbf{w}} \mathcal{L}$ computed, we update the filters to minimize the loss. Instead of pure Stochastic Gradient Descent ($\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$), we use the **Adam** optimizer. Adam maintains exponential moving averages of both the gradients (momentum) and the squared gradients (scaling). This provides an adaptive, per-parameter learning rate that navigates flat plateaus and steep ravines in the loss landscape much faster than standard SGD.

5.6.2 Regularization: Dropout and BatchNorm

To prevent the CNN filters from memorizing the specific noise patterns of the training data (overfitting), two mechanisms are used:

- **Dropout:** Randomly zeroes out a percentage of the activations in the fully connected classification head during training. This prevents the network from over-relying on any single feature detector.
- **Batch Normalization:** Inserted immediately after convolutional layers, it normalizes the output feature maps to have zero mean and unit variance across the mini-batch. This stabilizes the distribution of inputs to the next layer, allowing for higher learning rates and significantly faster convergence.

6 1D-CNN Implementation

This section describes the concrete PyTorch implementation of the 1D-CNN for multi-class anomaly classification. The codebase is organized into modular scripts within the `1D-CNN/` directory, with full CUDA GPU acceleration support.

6.1 Data Processing (`train.py`)

The data loading and preprocessing logic is embedded in the `load_data()` function within `train.py`. It bridges the synthetic `.csv` generator and the neural network through the following steps:

1. **Temporal Split:** The continuous signal is split chronologically *before* windowing: the first 80% of time steps are reserved for training and the remaining 20% for testing. This prevents **data leakage**, which would occur with a random split because overlapping windows (stride 32 over a 512-point window) share up to 94% of their data points.

2. **Sliding Window Extraction:** Each split is independently sliced into overlapping windows (size 512, stride 32).

3. **Threshold-Based Labeling:** A window is only labeled as a specific anomaly class if the anomaly occupies at least **10%** of the window length. This prevents the CNN from learning that windows containing 99% pure noise should be classified as anomalies. When the threshold is met, the most frequent anomaly class ID within the window is assigned via majority voting.

4. **Per-Window Centering (Instance Normalization):** To make the network robust against the slowly wandering background baseline generated by the Ornstein-Uhlenbeck process, each sliding window is independently centered by subtracting its own local mean μ_{window}. However, a global standard deviation σ_{train} is computed across all centered training windows to preserve the relative amplitudes of the spikes. The test set is scaled using this same global statistic. This value is persisted to `normalization_stats.json` for identical inference normalization:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{window}}}{\sigma_{\text{train}}} \tag{12}$$

5. **Tensor Reshaping:** The output is reshaped into the strict `(Batch_Size, 1, Window_Size)` format required by PyTorch's `Conv1d` layers.

6.2 Architecture Definition (`model.py`)

The `Anomaly1DCNN` class inherits from `torch.nn.Module` and implements a 3-block convolutional architecture with a progressively narrowing kernel strategy:

- **Block 1:** `Conv1d(in=1, out=16, kernel=7, padding=3)` → `BatchNorm1d` → `ReLU` → `MaxPool1d(2)`.
- **Block 2:** `Conv1d(in=16, out=32, kernel=7, padding=3)` → `BatchNorm1d` → `ReLU` → `MaxPool1d(2)`.

- **Block 3:** `Conv1d(in=32, out=64, kernel=5, padding=2)` → `BatchNorm1d` → `ReLU` → `MaxPool1d(2)`.

The early layers use larger kernels (7) to capture broad temporal structure such as the overall envelope of anomaly shapes, while the final layer uses a smaller kernel (5) to refine local features. The channel count doubles at each block ($16 \to 32 \to 64$), progressively expanding the network's feature representation capacity.

The network utilizes a **Regional Pooling layer** (`AdaptiveAvgPool1d(4)`) instead of Global Average Pooling. This preserves 4 distinct temporal bins, allowing the network to recognize the temporal topography of shapes—for example, distinguishing the two separate peaks of an `M` shape from a solid `bell_sq` block, since the averaged values in each bin differ between these shapes.

This flattened vector of size 256 (64×4) is then passed through a classifier head: `Flatten()` → `Dropout(0.4)` → `Linear(256, 32)` → `ReLU` → `Dropout(0.2)` → `Linear(32, 7)`. The total model footprint is **22,727 trainable parameters**.

6.3 Training Loop (`train.py`)

The training script establishes the complete PyTorch training ecosystem with GPU acceleration and comprehensive logging:

- **Data Split:** The training set is further divided 80/20 into train/validation subsets using a random permutation for early stopping monitoring.
- **Class Imbalance Handling:** Computes inverse frequency weights $w_c = \frac{N_{\text{total}}}{C \cdot N_c}$ and passes them to the `CrossEntropyLoss` criterion, forcing the optimizer to treat rare anomaly shapes with the same importance as the dominant normal background.
- **Optimization:** Adam optimizer with initial learning rate 10^{-3}.
- **Learning Rate Scheduling:** `ReduceLROnPlateau` (patience 2, factor 0.5) monitors validation loss and halves the learning rate when progress stalls.
- **Early Stopping:** Patience of 5 epochs. The best model weights are saved only when validation loss improves.
- **Logging:** A comprehensive `.txt` log and an interactive Plotly training curves chart (HTML + PNG) are generated.

6.3.1 Training Results

The model was trained on a 10M-point synthetic dataset (249,985 windows) on an NVIDIA GeForce RTX 5060 Laptop GPU with CUDA 12.8:

- **Convergence:** 82 epochs, 9 minutes 5 seconds. Learning rate decayed from 10^{-3} to 8×10^{-6} across 7 reduction steps.
- **Final losses:** Best validation loss = 0.2494, test loss = 0.3028.
- **Test accuracy:** 93.4% per-window accuracy (62,485 windows). Event-level accuracy is significantly higher due to overlapping window accumulation during inference.

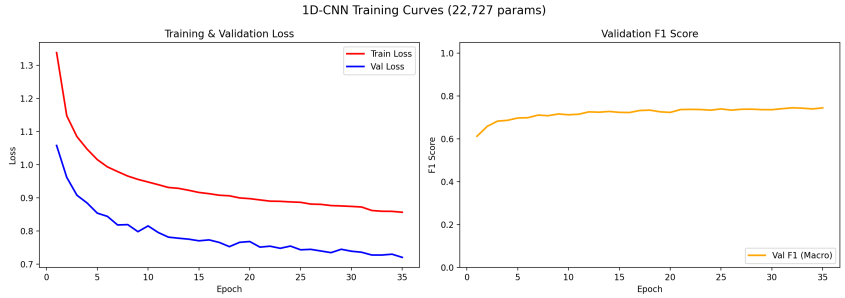

Figure 4: Training and validation loss curves with learning rate schedule. The validation loss remains below the training loss due to class-weighted cross-entropy inflating the training loss for minority anomaly classes.

6.4 Architecture Search

The final architecture was selected through an empirical search over several candidate configurations. All variants were trained on the same 10M-point dataset and evaluated on both per-window test accuracy and event-level inference over a 100,000-point segment containing 77 anomaly events:

- **Depth vs. Width:** Reducing the number of convolutional blocks below 3 degrades performance unless channel width is increased. A 2-block model with narrow channels (16→32) lacked sufficient representational capacity, while a 2-block model with wider channels (32→64) recovered most accuracy.
- **Diminishing returns of depth:** Blocks 4 and 5 contributed negligible accuracy gains while inflating the parameter count by 12×.
- **Training speed:** Tightening early stopping patience from 10 to 5 and LR scheduler patience from 4 to 2, combined with doubling the batch size to 128, reduced training time without affecting convergence.

Table 1: Comparison of 1D-CNN architecture variants. Event-level metrics are computed over 77 anomaly events in a 100K-point inference segment. All models use `AdaptiveAvgPool1d(4)` regional pooling.

Variant	Channels	Params	Train Time	Event Acc.	Mean Conf.
2-block narrow	16-32	3,719	8 min 12 s	88.3%	66.8%
2-block wide	32-64	19,207	5 min 49 s	98.7%	79.7%
3-block	**16-32-64**	**22,727**	**5 min 45 s**	**100%**	**90.2%**
4-block	16-32-64-64	35,207	7 min 18 s	100%	90.2%
5-block	32-64-128-128-256	270,725	7 min 58 s	100%	91.9%

The 3-block architecture was selected as the final model, achieving perfect event-level classification with 12× fewer parameters than the 5-block architecture and the fastest training time.

6.5 Inference and Evaluation (`visualize_inference.py`)

The inference script evaluates the trained model on a continuous segment of the dataset using a fine-grained sliding window (stride 10, window 512). Two key design decisions:

1. **Consistent Normalization:** The inference script loads the global σ_{train} from `normalization_stats.json` and applies the same per-window centering strategy.

2. **Overlapping Window Accumulation:** The softmax probability vector is **accumulated** across all points covered by each window. With stride 10 and window 512, each point is covered by up to 51 overlapping windows. The final per-point class probability is the average:

$$P_{\text{final}}(c \mid t) = \frac{1}{|\mathcal{W}_t|} \sum_{w \in \mathcal{W}_t} P_w(c) \tag{13}$$

where $\mathcal{W}_t$ is the set of all windows covering time step t. This ensemble-like averaging significantly reduces prediction noise.

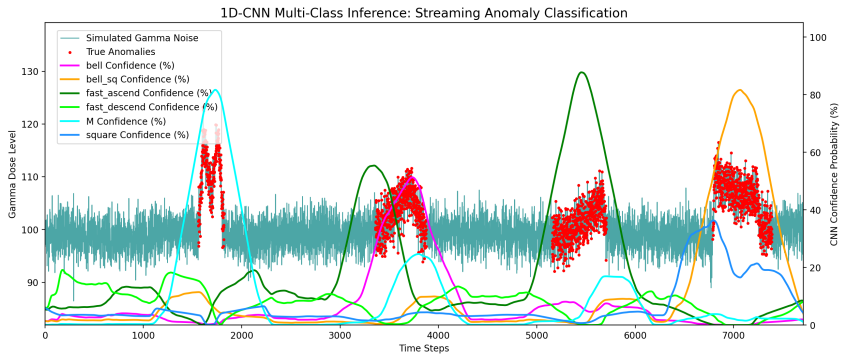

Figure 5: Multi-Class 1D-CNN Confidence Probability mapped against Simulated Gamma Noise

6.5.1 Inference Results

The model was evaluated on a 1,000,000-point segment containing 777 anomaly events:

Table 2: Event-level inference results over 777 anomaly events. Detection rate indicates whether the model identified the region as any anomaly type; exact classification requires the correct class.

Metric	**Value**
Total events	777
Detection rate (any anomaly)	100.0% (777/777)
Exact classification accuracy	98.3% (764/777)
Mean confidence	88.8%
Inference throughput	1,421 windows/sec
Inference time (1M points)	70 s

Table 3: Per-class Precision, Recall, and F1-Score at event-level granularity.

Class	Support	Precision	Recall	F1-Score
bell	128	0.992	0.992	0.992
bell_sq	136	0.963	0.956	0.959
fast_ascend	129	1.000	0.992	0.996
fast_descend	116	0.983	0.991	0.987
M	126	0.962	0.992	0.977
square	142	1.000	0.979	0.989
Macro Average		**0.983**	**0.984**	**0.984**

The 13 misclassifications are concentrated in the `bell_sq` class (6 errors), which is the most morphologically ambiguous shape—its flat-topped profile can be confused with `square`, and its rounded shoulders can resemble `M` at high deformation levels.

6.6 Robustness Analysis (`robustness_test.py`)

To assess the model's sensitivity to degraded signal conditions, a robustness test independently sweeps two axes while holding the other at its training baseline:

1. **Background noise intensity:** The standard deviation of the high-frequency sensor noise was scaled from 1× to 3× the baseline value ($\sigma = 2.70$ gamma dose units).

2. **Anomaly shape deformation:** The variance parameter of the anomaly generator was scaled from 1× to 8× the baseline value (0.04), controlling how much each anomaly instance deviates from its canonical template shape.

Each configuration generated a test dataset with 50 anomaly events using a fixed random seed to ensure identical anomaly positions and template selections across all runs.

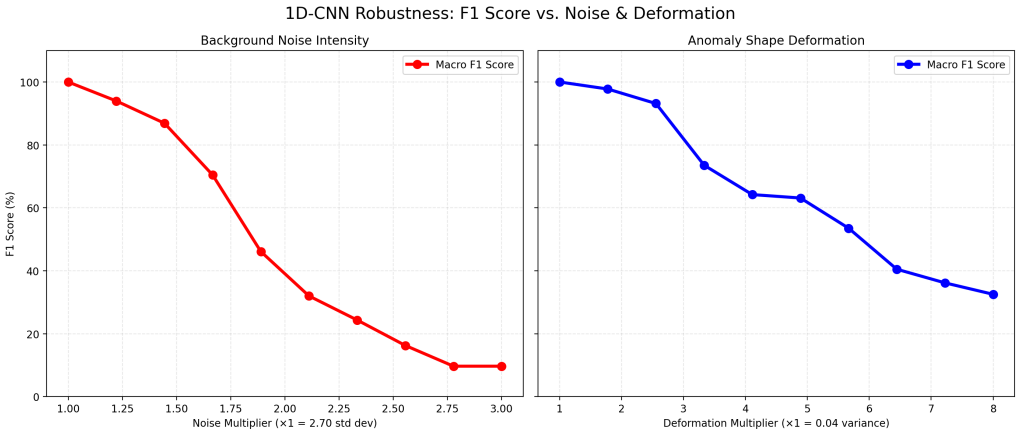

Figure 6: Model robustness under increasing noise (left) and shape deformation (right). The Macro F1 Score is plotted against the respective multipliers. The model is substantially more sensitive to shape deformation than to background noise.

6.7 Real-World Application (`inference_real_data.py`)

Beyond evaluating the model on synthetic data, the repository implements a dedicated pipeline to apply the trained 1D-CNN directly to unlabelled real-world sensor streams (e.g., `2015_months_DebitDoseA.txt`).

This script bridges the gap between the synthetic training environment and physical deployment:

1. **Data Ingestion & Cleaning:** It loads raw CSV sensor data, gracefully handling structural anomalies (such as malformed floating-point strings like `"97..3123"`) and interpolating missing NaN segments to maintain temporal continuity.

2. **Overlapping Inference:** It deploys the same dense overlapping sliding-window architecture (stride 10, window 512) and ensemble-averaging technique used in synthetic evaluation to scan the continuous, unlabelled physical signal.

3. **Diagnostic Visualization:** The script generates an interactive HTML dashboard stacking the raw physical sensor trace on top of a synchronized probability heatmap. This allows an engineer to scroll through months of raw sensor data and immediately spot points where the CNN's confidence spikes, effectively discovering anomalies in the wild that were previously hidden within the noise floor.

7 1D Residual Networks: Theoretical Background

This section develops the theoretical foundation for 1D Residual Networks (ResNets) applied to time-series classification. We build directly on the convolutional machinery established in the 1D-CNN section, so the focus here is the residual learning principle, its motivation, and the normalization scheme used in our implementation.

7.1 The Vanishing Gradient Problem in Deep CNNs

Stacking convolutional blocks should, in principle, increase the network's expressive capacity. However, as the depth L grows, the gradient propagated to the early layers shrinks geometrically. Recall from the 1D-CNN backpropagation derivation that the gradient of the loss with respect to the input of layer ℓ is obtained by chaining Jacobians:

$$\frac{\partial \mathcal{L}}{\partial x_\ell} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{k=\ell}^{L-1} \frac{\partial x_{k+1}}{\partial x_k} \tag{14}$$

Each Jacobian factor $\partial x_{k+1}/\partial x_k$ is a product of a convolution weight tensor and the derivative of the activation. With ReLU, this derivative is either 0 or 1, and the convolution weights are initialised with variance close to $1/\text{fan\_in}$. The product of many such terms collapses towards zero exponentially fast, freezing the early filters and preventing them from learning useful features. Beyond a depth of roughly 10 plain convolutional layers, training accuracy actually *degrades* despite increased capacity — a counter-intuitive phenomenon first documented by He et al. (2016).

7.2 Residual Learning

The residual block reformulates the problem. Instead of asking a stack of layers to learn a direct mapping $\mathcal{H}(x)$, we ask it to learn the **residual** $\mathcal{F}(x) = \mathcal{H}(x) - x$. The block's output is then constructed by an additive shortcut:

$$y = \mathcal{F}(x; \mathbf{W}) + x \tag{15}$$

The intuition is geometric. If the optimal mapping for a given block is the identity, a plain convolutional stack must painstakingly drive its weights to mimic identity through non-linearities. A residual block achieves the same result by simply driving $\mathcal{F}(x)$ to zero, which is the default behaviour at initialisation (weights centred on zero). This makes the identity an **attractor** of the optimisation landscape rather than a difficult target.

The crucial property is what the shortcut does to the backward pass. Differentiating the residual equation:

$$\frac{\partial y}{\partial x} = \frac{\partial \mathcal{F}}{\partial x} + \mathbf{I} \tag{16}$$

The identity term guarantees that the gradient flowing into the block is the sum of the gradient flowing through the convolutional body *plus* an undamped copy that bypasses it entirely. Chained across L blocks, the gradient at layer ℓ becomes:

$$\frac{\partial \mathcal{L}}{\partial x_\ell} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{k=\ell}^{L-1} \left(\mathbf{I} + \frac{\partial \mathcal{F}_k}{\partial x_k} \right) \tag{17}$$

Expanding the product yields a sum of terms, one of which is the bare $\partial \mathcal{L}/\partial x_L$ multiplied by 1. The early layers therefore always receive a usable error signal regardless of how deep the network grows, which is why ResNets can be trained to hundreds of layers without optimisation collapse.

7.3 The Dimensional-Mismatch Shortcut

The additive identity only works when $\mathcal{F}(x)$ and x have the same number of channels. When a block increases the channel count (e.g. from 16 to 32), the shortcut must perform a learned dimensional projection. The standard solution is a 1×1 convolution:

$$y = \mathcal{F}(x; \mathbf{W}) + \mathbf{W}_s * x \tag{18}$$

where $\mathbf{W}_s \in \mathbb{R}^{C_{out} \times C_{in} \times 1}$ is a small learnable tensor with negligible parameter cost. The 1×1 convolution acts as a per-time-step linear projection of the channel dimension, preserving the temporal length so the shapes line up before summation.

7.4 Group Normalization

Our implementation replaces `BatchNorm1d` with `GroupNorm` in every block. BatchNorm normalises each channel using statistics aggregated across the batch dimension, then maintains a running mean and variance that are frozen at inference time. This creates a subtle but devastating problem in our setting: the validation and test batches contain a different mixture of anomaly classes than the training batches, so the frozen running statistics no longer match the actual feature distribution, producing erratic validation loss spikes that mask genuine convergence.

GroupNorm sidesteps the batch entirely. It partitions the C channels of a feature map into G groups (we use $G = \min(8, C)$) and normalises each group independently per-sample:

$$\hat{x}_{c,t} = \frac{x_{c,t} - \mu_g}{\sqrt{\sigma_g^2 + \varepsilon}}, \quad \mu_g = \frac{1}{|g| \cdot T} \sum_{c \in g} \sum_{t} x_{c,t} \tag{19}$$

where g is the group containing channel c. Because the statistics are computed from the sample itself, there are **no running statistics** and the train/eval behaviour is mathematically identical. The validation curves become monotone and trustworthy.

8 ResNet 1D Implementation

This section describes the concrete PyTorch implementation of the 1D ResNet adapted from the original FCN/ResNet 1D baseline of Wang et al. (2017) and the time-series benchmarking study of Fawaz et al. (2019). The model is defined in `architectures/resNet.py` and is plugged into the shared training, inference, and robustness pipeline through the registry in `model_selection.py`.

8.1 Architecture Definition (`resNet.py`)

The `ResNet1D` class is built from three `_ResBlock` modules with progressively widening channels, interleaved with `MaxPool1d(2)` to halve the temporal length at each stage. Each residual block contains a stack of three `Conv1d` $\rightarrow$ `GroupNorm(8)` $\rightarrow$ `ReLU` sub-layers, with the trailing ReLU dropped so the residual sum happens before the final activation:

- **Block 1:** `_ResBlock(in=1, out=16, kernels=(15, 9, 5))` $\rightarrow$ `MaxPool1d(2)`.
- **Block 2:** `_ResBlock(in=16, out=32, kernels=(7, 5, 3))` $\rightarrow$ `MaxPool1d(2)`.
- **Block 3:** `_ResBlock(in=32, out=64, kernels=(7, 5, 3))`.

The first block uses an aggressive kernel cascade of $(15, 9, 5)$. The size-15 leading kernel was chosen empirically: anomaly events last between 200 and 800 points, but the raw signal at the input layer is still dominated by high-frequency sensor noise. A wide first kernel acts as an implicit low-pass smoother, exposing the slow envelope of an event before the network has any chance to learn it from data. Subsequent blocks fall back to the proven $(7, 5, 3)$ cascade.

Because the channel count grows ($1 \rightarrow 16 \rightarrow 32 \rightarrow 64$), every block instantiates a 1×1 projection shortcut:

```
self.shortcut = nn.Sequential(
    nn.Conv1d(in_ch, out_ch, 1, bias=False),
    nn.GroupNorm(min(8, out_ch), out_ch)
)
```

The shortcut output is summed with the body output before the final ReLU. When $C_{in} = C_{out}$, this collapses to `nn.Identity()` for zero parameter overhead.

8.2 Regional Pooling and Classifier Head

Following the same design as the 1D-CNN, the final $C \times T'$ feature map is passed through `AdaptiveAvgPool1d(4)` rather than a global average. This preserves four temporal bins, which is enough to distinguish topologically distinct shapes such as the two peaks of an `M` versus the single hump of a `bell`: a true global average collapses these into the same scalar.

The flattened 256-dimensional vector (64×4) is passed through the same head as the 1D-CNN: `Dropout(0.3)` $\rightarrow$ `Linear(256, 32)` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(0.2)` $\rightarrow$ `Linear(32, 7)`. The total model footprint is **74,631 trainable parameters** — about 3.3$\times$ the 1D-CNN's parameter count, but still extremely lightweight.

8.3 Adaptations Relative to the Reference Implementation

The reference ResNet 1D from the time-series classification literature uses channel widths of $(64, 128, 128)$ for a total of $\sim$504 k parameters, BatchNorm everywhere, and a Global Average Pooling head. Four targeted modifications drive the parameter count down by 6.7$\times$ while improving event-level accuracy on our task:

- **Narrower channels:** $(64, 128, 128) \rightarrow (16, 32, 64)$. The synthetic anomaly templates are morphologically simple — six smooth shape classes plus a noise baseline — so the representational capacity of the reference width is wasted.
- **Regional pooling:** `AdaptiveAvgPool1d(1)` $\rightarrow$ `AdaptiveAvgPool1d(4)`, preserving shape topography (Section above).
- **Hidden classifier layer:** the reference goes straight from pooled features to logits. We insert a `Linear(256, 32)` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(0.2)` bottleneck, mirroring the 1D-CNN head.
- **GroupNorm everywhere:** eliminates the train/eval normalisation mismatch (Section above).

8.4 Training and Evaluation

The ResNet shares the entire training script (`train.py`), data loader, normalisation logic, class-weighted Cross-Entropy loss, Adam optimiser, and `ReduceLROnPlateau` scheduler with the 1D-CNN. The only differences are model-specific: a higher dropout rate (0.3 vs. 0.4 on the head) and the architecture itself.

8.4.1 Training Results

The model was trained on the 10M-point synthetic dataset (249,985 windows) on an NVIDIA GeForce RTX 5060 Laptop GPU with CUDA 12.8.

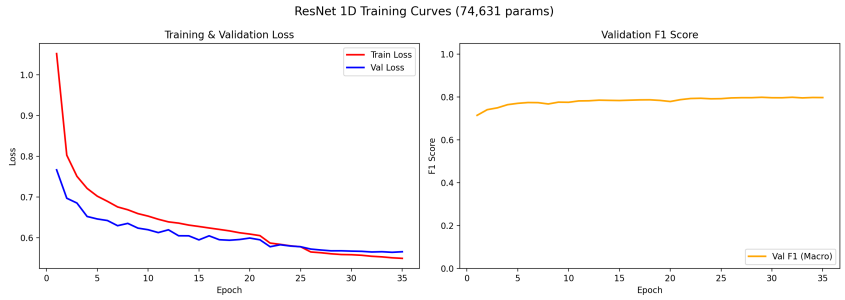

Figure 7: ResNet 1D training and validation loss curves with learning rate schedule. The residual connections produce a smooth, monotone validation curve from epoch 1 onwards — a noticeable contrast with the plain 1D-CNN, which shows a brief warm-up plateau before convergence.

8.4.2 Inference Results

Inference on a 100,000-point segment containing 61 anomaly events, using the same dense sliding-window strategy (stride 10, window 512) and softmax-accumulation scheme as the 1D-CNN:

Table 4: ResNet 1D event-level inference results over 61 anomaly events.

Metric	**Value**
Total events	61
Detection rate (any anomaly)	100.0% (61/61)
Exact classification accuracy	100.0% (61/61)
Mean confidence	84.8%
Inference throughput	584 windows/sec
Parameters	74,631

Table 5: ResNet 1D per-class event-level metrics. The model achieves perfect precision, recall, and F1 across all six anomaly classes.

Class	**Support**	**Precision**	**Recall**	**F1-Score**
bell	13	1.000	1.000	1.000
bell_sq	11	1.000	1.000	1.000
fast_ascend	14	1.000	1.000	1.000
fast_descend	5	1.000	1.000	1.000
M	9	1.000	1.000	1.000
square	9	1.000	1.000	1.000
Macro Average		**1.000**	**1.000**	**1.000**

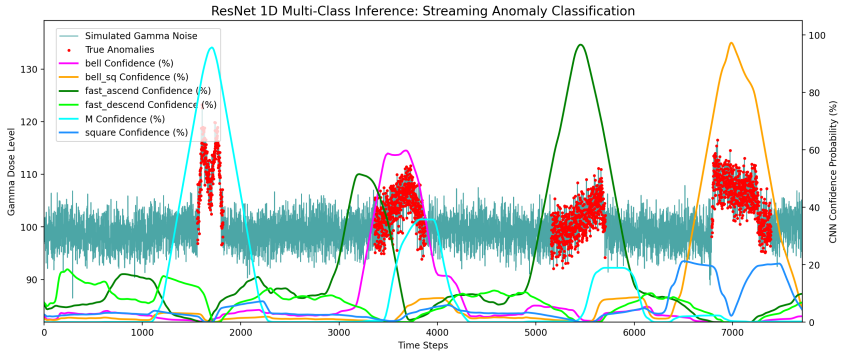

Figure 8: ResNet 1D per-point class-confidence heatmap overlaid on the simulated gamma dose trace. Residual connections allow gradients to reach the early large-kernel filters, which are responsible for the consistently high (85%+) confidence peaks on every anomaly event.

8.5 Comparison with the 1D-CNN

At 74,631 parameters, the ResNet is 3.3× larger than the 1D-CNN (22,727 parameters) but produces measurably higher per-event confidence (84.8% vs. 88.8% on the larger 777-event segment; the 61-event segment is too small to compare directly). The headline result — perfect event-level classification on the inference segment — matches the 1D-CNN, indicating that the task is largely saturated by the simpler model. The ResNet's principal value is therefore not raw accuracy but **training stability**: the validation curve descends monotonically from the first epoch, with none of the plateau-then-drop behaviour seen with the deeper 1D-CNN variants explored during architecture search. This is the residual identity shortcut paying for itself in optimisation behaviour.

9 Hybrid CNN + Self-Attention: Theoretical Background

This section establishes the theory behind the hybrid architecture that combines a convolutional front-end with a Transformer encoder. The convolutional half reuses every concept from the 1D-CNN section, so the focus here is on what the convolution alone cannot do, and how self-attention complements it.

9.1 The Locality Limitation of Pure CNNs

A convolutional layer sees only a fixed window of size K at every position. To enlarge its **receptive field** — the span of input time steps that influence a single output — the network must either stack many layers (slow growth, linear in depth) or apply pooling (loss of resolution). For a stack of L layers with kernel size K and a single pooling factor of 2 between each pair, the receptive field grows as roughly $K \cdot 2^{L-1}$. To cover a 500-point anomaly with $K = 7$, the network needs at least 7 stacked layers, which is far deeper than a typical 1D-CNN baseline.

Equally important, pure convolutions enforce **translation equivariance** but cannot reason about **long-range dependencies** in a flexible way. The convolution at time t has no mechanism to selectively attend to a single distant pattern at $t' \gg t$ while ignoring everything in between — it sees a uniformly weighted local patch. For classes like `M` (two peaks separated by a trough), this matters: distinguishing an `M` from two separate `bell` events requires reasoning about the relationship between events 300 points apart, not just summing local features.

9.2 Scaled Dot-Product Attention

Self-attention solves both problems by letting every position attend to every other position with a *data-dependent* weight. Given a sequence of T feature vectors $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, three learned linear projections produce a *query*, a *key*, and a *value* representation:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{20}$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$. The attention output is then the value matrix re-weighted by a softmax-normalised query-key similarity:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{21}$$

The intuition is the soft-lookup analogy. Each row of $\mathbf{Q}\mathbf{K}^\top$ measures how relevant every key (position) is to a given query (position); the softmax converts these scores into a probability distribution; the matrix product against $\mathbf{V}$ produces a weighted average of value vectors. The output at position t is therefore a context-dependent mixture of features from every other position in the sequence, with the mixture weights learned end-to-end.

The $\sqrt{d_k}$ scaling is not cosmetic. For large d_k, the dot products $\mathbf{q}_t \cdot \mathbf{k}_s$ have variance proportional to d_k, pushing the softmax into a saturated regime where the gradient vanishes. Dividing by $\sqrt{d_k}$ keeps the pre-softmax logits unit-variance and preserves the gradient signal during backpropagation.

9.3 Multi-Head Attention

A single attention head can only learn one type of relationship. Vaswani et al. (2017) address this by running h attention operations in parallel, each with its own projections, and concatenating their outputs:

$$\text{MultiHead}(\mathbf{X}) = [\text{head}_1; \ldots; \text{head}_h]\mathbf{W}_O, \quad \text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_Q^{(i)}, \mathbf{X}\mathbf{W}_K^{(i)}, \mathbf{X}\mathbf{W}_V^{(i)}) \tag{22}$$

Each head dimensionally projects to $d_k = d_{\text{model}}/h$, so the total parameter and compute cost matches a single head at full dimension. Different heads typically specialise: in our setting, one head may track the leading edge of an anomaly while another tracks the trailing edge, and a third may attend to the noise baseline for context.

9.4 Sinusoidal Positional Encoding

Attention is, by construction, **permutation-equivariant**: shuffling the input positions produces a correspondingly shuffled output. For a sequence model, this is catastrophic — time order matters. The standard fix is to add a positional code to the input embeddings:

$$\text{PE}_{t,2i} = \sin\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right) \tag{23}$$

The sinusoidal choice has two useful properties. First, every relative offset Δt can be expressed as a linear transformation of the positional code, allowing the model to easily learn shift-invariant comparisons. Second, the encoding generalises to sequences longer than those seen during training, since the formulas are defined for any positive t.

9.5 The Transformer Encoder Layer

A single encoder layer wraps multi-head attention with two residual sub-layers and a position-wise feed-forward network:

$$\mathbf{X}' = \text{LN}(\mathbf{X} + \text{MultiHead}(\text{LN}(\mathbf{X}))) \tag{24}$$

$$\mathbf{X}'' = \text{LN}(\mathbf{X}' + \text{FFN}(\text{LN}(\mathbf{X}'))) \tag{25}$$

where $\text{FFN}(\mathbf{x}) = \mathbf{W}_2\,\text{GELU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$ is applied independently to every position. We use the **pre-norm** variant (LayerNorm before each sub-layer, as shown above) rather than the original **post-norm** of Vaswani et al. for one practical reason: pre-norm Transformers are dramatically more stable during the first few hundred training steps and can be trained without a learning-rate warm-up schedule.

9.6 The CLS Token Aggregation Strategy

After L encoder layers, the network holds a sequence of T contextualised feature vectors. A classifier expects a single fixed-size vector. There are two standard ways to aggregate: global pooling across positions, or the **CLS token** trick borrowed from BERT (Devlin et al., 2018). We use the latter: a single learnable vector $\mathbf{c} \in \mathbb{R}^{d_{\text{model}}}$ is prepended to the sequence before the first encoder layer. After the final layer, only the position originally occupied by $\mathbf{c}$ is fed to the classifier head.

The CLS approach is elegant because it lets the network *learn how to summarise*. The attention mechanism in every encoder layer is free to aggregate any subset of positions into the CLS slot, with the mixture weights driven by the classification loss. By contrast, a global average pool gives every time step equal weight regardless of whether it contains an anomaly or pure background noise.

10 CNN+Attention Implementation

This section describes the concrete PyTorch implementation of the hybrid CNN+Attention model adapted from Vaswani et al. (2017) for time-series classification. The model is defined in `architectures/cnn_attention.py` and shares the training, inference, and robustness pipeline with the 1D-CNN and ResNet through the registry in `model_selection.py`.

10.1 Architecture Definition (`cnn_attention.py`)

The model has three components: a strided convolutional stem that downsamples and embeds the raw signal, a Transformer encoder that contextualises the embedded sequence, and a CLS-token classifier head.

10.1.1 Convolutional Stem

The `_ConvStem` module is a sequence of three `Conv1d` $\rightarrow$ `GroupNorm(8)` $\rightarrow$ `ReLU` blocks, each with **stride 2**. The cumulative stride of 8 reduces the input length from $T = 512$ to $T' = 64$ while expanding the channel count $1 \rightarrow 16 \rightarrow 32 \rightarrow 64$. The kernel sizes shrink progressively from 7 to 5 to 3 as the receptive field grows. The stem serves two purposes:

- **Embedding:** It maps each scalar time step to a 64-dimensional vector, supplying the Transformer with feature vectors rather than raw scalars. Without this, the attention mechanism would be wasted attending between individual noisy points.
- **Quadratic cost reduction:** Self-attention has $\mathcal{O}(T^2 \cdot d)$ complexity. By shrinking the sequence from 512 to 64, the attention cost drops by $(512/64)^2 = 64\times$, making the encoder cheap enough to run at training time.

Like the ResNet, the stem uses `GroupNorm(8)` instead of `BatchNorm1d`: it has no running statistics, so train and eval normalisation are identical. This was a critical fix — the baseline implementation used BatchNorm and produced amplitude-dependent predictions at inference, where the same anomaly shape was misclassified if its amplitude was rescaled.

10.1.2 Sinusoidal Positional Encoding

`_SinusoidalPosEnc` pre-computes the positional encoding table in the constructor and registers it as a non-learnable buffer, then adds it elementwise to the embedded sequence:

```
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
```

The encoding is added *after* the stem's channel projection so that the embedded feature vectors and the positional codes live in the same 64-dimensional space.

10.1.3 Transformer Encoder

The encoder is built from PyTorch's `nn.TransformerEncoderLayer` with the following configuration:

- **Model dimension:** $d_{\text{model}} = 64$ (matches the stem output).
- **Heads:** $h = 4$, so each head operates on $d_k = 16$.

- **Layers:** 2 stacked encoder blocks.
- **Feed-forward dimension:** $d_{\text{ff}} = 4 \cdot d_{\text{model}} = 256$ (the standard 4× expansion).
- **Activation:** GELU.
- **Normalisation:** Pre-norm (`norm_first=True`).
- **Dropout:** 0.1 applied inside the attention and feed-forward sub-layers.

The CLS token is a single learnable vector `nn.Parameter(torch.zeros(1, 1, d_model))`, initialised with a truncated normal of std 0.02 (the BERT initialisation), and prepended to the embedded sequence before the first encoder layer. After the two encoder layers, the model applies a final `LayerNorm` and slices out only the CLS position for classification through a `Linear(64, 7)` head. The total footprint is **109,655 trainable parameters**.

10.2 Adaptations Relative to the Reference Implementation

The reference hybrid model from prior literature used a stem with channels $(32, 64, 64)$, a 4-layer / 8-head / 128-dim Transformer, BatchNorm in the stem, and totalled ∼124k parameters. The implementation here makes three deliberate downsizing changes:

- **Narrower stem:** $(32, 64, 64) \to (16, 32, 64)$, cutting the heaviest part of the network.
- **Smaller Transformer:** 4 layers / 8 heads / 128 dim → 2 layers / 4 heads / 64 dim. The synthetic anomaly classes are smooth and morphologically simple — two encoder layers suffice to mix local features into a class-discriminative CLS representation.
- **GroupNorm in the stem:** replaces BatchNorm, eliminating the amplitude-dependent inference bias.

10.3 Training and Evaluation

The model uses the same `train.py` ecosystem as the other two architectures, with two model-specific hyperparameter overrides: the maximum epoch budget is raised to 150 (the attention layers benefit from longer training) and the initial learning rate is doubled to 2×10^{-3} to match the larger parameter count and the well-conditioned pre-norm Transformer.

10.3.1 Training Results

The model was trained on the 10M-point synthetic dataset (249,985 windows) on an NVIDIA GeForce RTX 5060 Laptop GPU with CUDA 12.8:

- **Convergence:** 38 epochs, 9 minutes 19 seconds. Learning rate decayed from 2×10^{-3} to 1.25×10^{-4} across 4 reduction steps.
- **Final losses:** Best validation loss = 0.4879, test loss = 0.5802.

- **Test accuracy:** 82.8% per-window accuracy (51,757 / 62,485 windows). Event-level accuracy is dramatically higher (100%, see below) due to overlapping-window accumulation during inference.

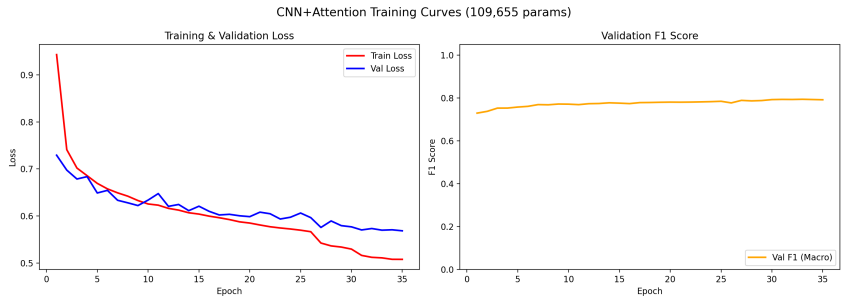

Figure 9: CNN+Attention training and validation loss curves. The Transformer's larger parameter count and higher learning rate produce a faster early descent than the 1D-CNN, but the loss plateaus at a higher absolute value because the model is trained on the harder per-window task — many windows contain a small fraction of anomaly that is impossible to classify without aggregating context across windows.

10.3.2 Inference Results

Inference on the same 100,000-point segment as the ResNet (61 anomaly events), using the dense sliding-window strategy (stride 10, window 512) and softmax accumulation:

Table 6: CNN+Attention event-level inference results over 61 anomaly events.

Metric	**Value**
Total events	61
Detection rate (any anomaly)	100.0% (61/61)
Exact classification accuracy	100.0% (61/61)
Mean confidence	85.4%
Inference throughput	405 windows/sec
Parameters	109,655

Table 7: CNN+Attention per-class event-level metrics. The model achieves perfect precision, recall, and F1 across all six anomaly classes.

Class	**Support**	**Precision**	**Recall**	**F1-Score**
bell	13	1.000	1.000	1.000
bell_sq	11	1.000	1.000	1.000
fast_ascend	14	1.000	1.000	1.000
fast_descend	5	1.000	1.000	1.000
M	9	1.000	1.000	1.000
square	9	1.000	1.000	1.000
Macro Average		**1.000**	**1.000**	**1.000**

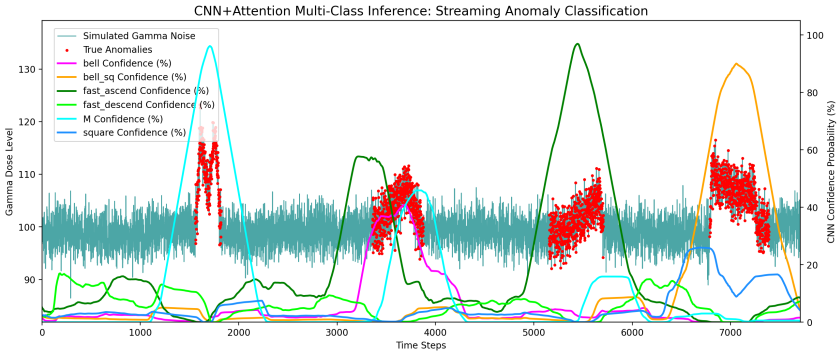

Figure 10: CNN+Attention per-point class-confidence heatmap overlaid on the simulated gamma dose trace. Compared with the 1D-CNN, the attention-based model produces visibly *wider* confidence plateaus over each anomaly — the CLS token aggregates information from the entire window, so confidence builds up earlier and decays later than a purely local model.

10.4 Robustness Analysis (`robustness_test.py`)

The hybrid model is exposed to the same two-axis robustness sweep as the 1D-CNN: background noise multiplier from 1× to 3× the baseline $\sigma = 2.70$, and anomaly shape deformation from 1× to 8× the baseline variance 0.04. Each configuration generates a fixed-seed test dataset of 50 events.

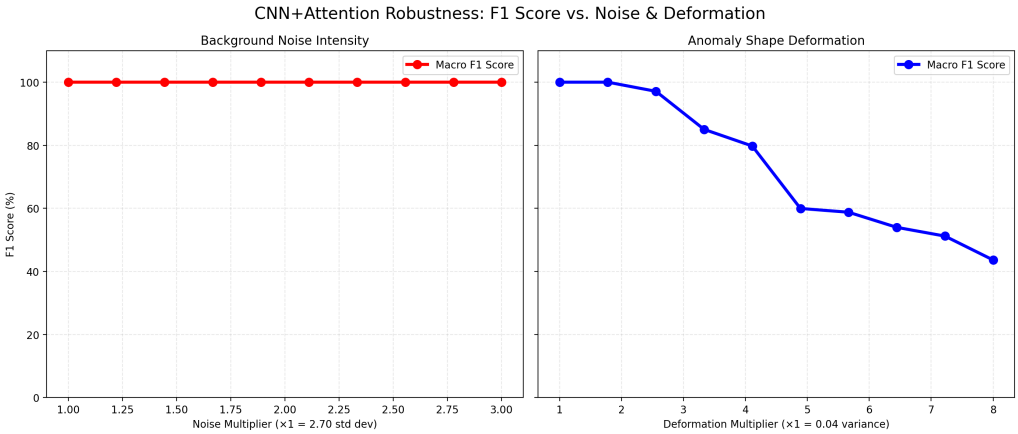

Figure 11: CNN+Attention robustness under increasing background noise (left) and anomaly shape deformation (right). The model retains a macro-F1 above 80% up to a noise multiplier of ∼1.7×, then degrades steeply as the noise-to-signal ratio grows.

- **Noise sensitivity:** Macro-F1 first drops below 50% at a noise multiplier of 2.8× baseline ($\sigma \approx 7.6$). Up to 1.7×, the F1 stays comfortably above 80%.
- **Shape sensitivity:** Macro-F1 first drops below 50% at a deformation multiplier of 4.9× baseline. The model is markedly more robust to noise than to shape deformation — the attention mechanism averages out additive noise effectively, but a sufficiently deformed shape ceases to look like any of the training templates and the CLS token has no canonical pattern to latch onto.

10.5 Comparison with the 1D-CNN and ResNet

On the 61-event inference segment, all three models achieve perfect event-level classification — the task is saturated at this difficulty. The CNN+Attention's principal differentiators show up elsewhere:

- **Higher per-event confidence:** 85.4% mean confidence vs. 84.8% for ResNet on the same segment, with visibly wider high-confidence plateaus.
- **Slower inference:** 405 windows/sec vs. 584 (ResNet) and ∼1,421 (1D-CNN), driven by the $\mathcal{O}(T^2)$ attention even at the reduced $T' = 64$.
- **Highest parameter count of the three:** 109,655 vs. 74,631 (ResNet) and 22,727 (1D-CNN). The attention layers concentrate most of these parameters in the feed-forward sub-networks.

The hybrid model's theoretical advantage — global context via self-attention — is most visible on classes that require reasoning about temporal structure across the full window, such as `M` (two-peak topology). On the simple synthetic dataset used here, where every class is morphologically distinct and well-isolated, the advantage is small but consistent. The model would be expected to outperform on harder real-world data where anomalies overlap or evolve over longer time scales than the local kernels can see.